GigBid — Demo & Viva Preparation Guide (DEMO_PREP)

Prepared for: Abdullah Razzaq · Roll No: 23i-0057 · Section B · AI-4016 Assignment 2

A. Project Overview: What Is Being Done in GigBid?

GigBid is a full-stack, human-in-the-loop **Multi-Agent Freelancer Copilot**. Instead of a single chatbot, it uses a team of specialized LangGraph agents that:
1. **Discover** freelance jobs from a Marketplace (GigSim simulator or live Upwork MCP) via a single swappable port.
2. **Analyze in parallel**: extract structured requirements (Job Analyst), detect scams/prompt injections (Risk Assessor), and match against your real portfolio via Advanced RAG + MCP (Fit Matcher).
3. **Price & Draft**: compute a fair market bid + Nov 2 Connects surcharge, draft a proposal citing [evidence_id]s, strip any unsupported claims (Claim Verifier), critique (max 2 loops), and check platform rules (Compliance Guard).
4. **Pause for Human Approval**: uses LangGraph interrupt() so NOTHING is ever submitted, messaged, or spent without your explicit click in the web UI.

B. How Everything Was Implemented & Connected Together

• **Marketplace Simulator (gigsim/):** FastAPI + in-memory/JSON store with 1,500 historical jobs, 600 dev jobs (including scams and prompt injections), 3 freelancer personas, and a hidden-preference client reply simulator.
• **MCP Layer (mcp_servers/):** 3 FastMCP servers — *portfolio* (read-only RAG search), *marketplace* (GigSim/Upwork behind MarketplacePort with idempotency_key, dry_run, rate limit, audit log), and *workspace* (calendar slots & interview holds). Agents only call tools via agent/mcp_client.py allowlists.
• **Advanced RAG (rag/):** 8 variants (V0–V9) + sentence-level Claim Verifier (rag/verify.py). Evaluation proved v3_rerank (Hybrid BM25+Dense RRF + Cross-Encoder Rerank) hits 1.000 Recall@5.
• **LangGraph (agent/graphs/):** bid_graph.py (parallel Send + Critic loop + interrupt + SqliteSaver durability) and conversation_graph.py (Client Liaison + interrupt).
• **Web UI & API (apps/api/ + apps/web/):** FastAPI backend with SSE streaming + React/TypeScript frontend (9 screens, Playwright E2E tests).

C. How to Run & Explain Every Possible Demo Scenario

Start Command: Run make demo (or uvicorn apps.api.main:app --port 8000) and open http://127.0.0.1:8000.

Scenario	Exact Clicks in UI	What to Say to TA
1. Login & Onboarding	Go to '/', click 'Sign In & Save Profile' (pre-filled: reviewer_gigbid_session, 2026)	Show persistent session, portfolio corpus count, and add/delete evidence
2. Inbox to Approved Proposal	Go to '/inbox', click 'Analyze & Draft' on a high-fit job -> click 'Open Studio' -> click 'Approve'	Point out Job Analyst, Risk Assessor, Fit Matcher, Bid Strategist, Proposal Writer, Claim Verifier, Compliance Guard, and Critic running in parallel
3. Edit Draft -> Style Memory	In Studio, edit the cover letter (e.g. remove 'passionate') -> click 'Save Edit & Approve'	Show the new 'Style Memory' settings: promotion_rules.py,
4. Scam & Injection Caught	Analyze a scam job or injection job in '/inbox'.	Show Risk Assessor flagging 'scam_or_unpaid_work' or 'prompt_injectio
5. Client Reply & Calendar	Go to '/messages' -> click 'Draft AI Reply & Interview Slots' -> 'Approve & Send Reply'	Show Client Liaison classifying 'interview_request' and fetching fr
6. Degraded Mode	Go to '/runs' -> click 'Simulate Marketplace MCP Offline' -> click 'Degraded'	Show the global Degraded Mode banner; restore it in '/runs' to recover.

D. Viva Questions (Page 14) — Exact Code Locations & Answers

#	Examiner Question	Exact Answer & File Reference
1	Kill Marketplace MCP server. What does user see & where is it handled?	User sees 'Degraded Mode' banner in InboxPage.tsx and work is queued. Handled in agent/
2	Approve proposal, crash worker before submit. Prove exactly-once.	Proof via PostgresSaver (agent/graphs/checkpointer.py). submit_proposal passes deter
3	Delete evidence behind a claim and regenerate. What happens?	Delete via DELETE /api/auth/portfolio/{id} (or Onboarding UI) -> click 'Regenerate & Re-verify' in
4	Open trace of a skipped job. Which node got it wrong?	Open /runs -> RunTimeline shows each node's model, tool, tokens, and observation. decision_r
5	Show prompt-injection defence & an attack that evaded it.	Defended: <untrusted_job_post> tags in analyst.py/risk_assessor.py + regex + Claim Verifier (0.0
6	What did Style Memory learn? Show promotion rule & where it lives.	Show agent/memory/promotion_rules.py (extract_style_rule_from_diff). Raw tool output never en
7	Switch GigSim to live mode in config. What changes & what doesn't?	Change marketplace_mode: upwork_live in agent/config.yaml. Only get_marketplace_adapter() i
8	Judge vs human labels disagreed. Show two and say who was right.	human_eval.json, n=30 calibration had Cohen's kappa=0.841 (28/30 agreed, 2 borderlines where
9	Where does Single ReAct beat your graph? Would you ship it?	Single ReAct is faster/cheaper on simple 1-requirement jobs, but had 0.680 unsupported claims/
10	How many lines changed to absorb Nov 2 Change Request?	3 config keys in agent/config.yaml (policy_nov2) + 6 lines in pricing.py & compliance.py (<10 line